

1. Namespaces in Hadoop

What is a Namespace?

A namespace in Hadoop is a hierarchical organization of files and directories in the Hadoop Distributed File System (HDFS). Think of it as the directory tree structure that holds all the metadata about files and directories.

Key Concepts

Namespace Management:

- The NameNode manages the namespace
- It maintains the entire file system namespace in memory
- All namespace operations (create, delete, modify) are recorded by the NameNode

Components of Namespace:

- Inodes: Represent files and directories with attributes like permissions, modification times, disk space quotas
- Block Map: Mapping between files and their data blocks
- Directory Structure: Hierarchical organization similar to traditional file systems

Namespace Features

Federation (Multiple Namespaces):

- Hadoop supports HDFS Federation where multiple NameNodes can manage independent namespaces
- Each NameNode manages a namespace volume
- Benefits: Scalability, isolation, and increased throughput

Namespace Operations:

Creating directories

Creating files

Deleting files/directories

Renaming files/directories

Setting permissions and ownership

Namespace Storage

FSImage:

- Snapshot of the entire file system namespace
- Stored on disk
- Contains inode data and file-to-block mapping

EditLog:

- Transaction log recording all changes to the namespace
- Every modification is first written to EditLog
- Used for recovery in case of failures

Namespace Quotas

Name Quota:

- Limits the number of file and directory names in the tree
- Command: `hdfs dfsadmin -setQuota <N> <directory>`

Space Quota:

- Limits the amount of disk space used
- Command: `hdfs dfsadmin -setSpaceQuota <N> <directory>`

The Simple Analogy: Replication vs. Erasure Coding

Imagine you have a 1 TB file you need to protect from disk failures.

1. Traditional HDFS Replication (The Old Way)

Hadoop's default method is to make three full copies of your file

- **How it works:** You have one original 1 TB file and two identical 1 TB copies.
- **Storage Cost:** 1 TB (Original) + 1 TB (Copy 1) + 1 TB (Copy 2) = **3 TB total**.
- **Overhead:** 200%
- **Fault Tolerance:** You can lose any two copies and your data is still safe

2. Erasure Coding (The Hadoop 3 Way)

Erasure Coding uses a "data plus parity" model It breaks the file into small chunks, then calculates a few smaller "parity" chunks

- **How it works:** With a common EC policy (RS-6-3), your 1 TB file is split into 6 data blocks The system then computes 3 parity blocks These 9 total blocks are stored on 9 different nodes.
- **Storage Cost:** 1 TB (Original Data) + 0.5 TB (Parity Data) = **1.5 TB total**.
- **Overhead:** 50%
- **Fault Tolerance:** You can lose any 3 of those 9 blocks, and the system can automatically reconstruct the lost data using the remaining blocks.

This approach provides the *same or better* fault tolerance (losing 3 nodes vs. 2) for *half* the storage cost.

Why Isn't Everything Erasure Coded? (The Trade-Offs)

While EC saves a massive amount of space, it comes with trade-offs, which is why Hadoop 3 supports both methods.

Feature	Replication (Hadoop 2 & 3)	Erasure Coding (Hadoop 3)
Storage Cost	Very High (200% overhead)	Low (~50% overhead)

CPU Usage	Very Low. Just simple file copies.	High. Requires complex math (encoding & decoding) to read or write data.
Network Usage	Low for reads. High for writes.	High. Rebuilding a lost block requires reading multiple blocks from across the network.
Best For...	"Hot" Data. Frequently accessed and modified files.	"Warm" or "Cold" Data. Large, older files that are read infrequently (e.g., archives, backups).

Key Limitations of Erasure Coding:

- **Higher CPU Load:** The encoding/decoding process is computationally expensive
- **Slower Recovery:** When a node fails, rebuilding the lost EC blocks (decoding) puts a heavier load on the CPU and network than simply re-copying a replicated block.
- **File Operations:** EC does not support certain file operations like append (adding data to the end of an existing file) or hflush (forcing data to disk)

3. Hadoop Commands - Portal Admin and Important Commands

HDFS Administration Commands (dfsadmin)

1. Report Cluster Status

```
hdfs dfsadmin -report
```

Shows: Available space, used space, DataNode status

2. Safe Mode Operations

Enter safe mode

```
hdfs dfsadmin -safemode enter
```

Leave safe mode

```
hdfs dfsadmin -safemode leave
```

Get safe mode status

```
hdfs dfsadmin -safemode get
```

Wait for safe mode to exit

hdfs dfsadmin -safemode wait

3. Save Namespace

Save current namespace to FSImage

hdfs dfsadmin -saveNamespace

4. Refresh Nodes

Refresh DataNode list

hdfs dfsadmin -refreshNodes

Refresh service ACLs

hdfs dfsadmin -refreshServiceAcl

5. Set/Clear Quotas

Set name quota (number of files/directories)

hdfs dfsadmin -setQuota 1000 /user/hadoop

Set space quota (in bytes)

hdfs dfsadmin -setSpaceQuota 10g /user/hadoop

Clear quotas

hdfs dfsadmin -clrQuota /user/hadoop

hdfs dfsadmin -clrSpaceQuota /user/hadoop

6. Decommission Nodes

Refresh nodes for decommissioning

hdfs dfsadmin -refreshNodes

Check decommission status

hdfs dfsadmin -report

HDFS File System Commands (dfs)

1. Basic File Operations

List files

hdfs dfs -ls /user/hadoop

hdfs dfs -ls -R /user/hadoop Recursive

Create directory

hdfs dfs -mkdir /user/hadoop/data

hdfs dfs -mkdir -p /user/hadoop/data/nested Create parent dirs

Copy from local to HDFS

```
hdfs dfs -put localfile.txt /user/hadoop/
```

```
hdfs dfs -copyFromLocal localfile.txt /user/hadoop/
```

Copy from HDFS to local

```
hdfs dfs -get /user/hadoop/file.txt ./
```

```
hdfs dfs -copyToLocal /user/hadoop/file.txt ./
```

Copy within HDFS

```
hdfs dfs -cp /source/file.txt /destination/
```

Move files

```
hdfs dfs -mv /source/file.txt /destination/
```

Delete files

```
hdfs dfs -rm /user/hadoop/file.txt
```

```
hdfs dfs -rm -r /user/hadoop/directory Recursive delete
```

2. File Information

Check file status

```
hdfs dfs -stat "%n %o %r %u %g %b" /user/hadoop/file.txt
```

Get file checksum

```
hdfs dfs -checksum /user/hadoop/file.txt
```

Display file content

```
hdfs dfs -cat /user/hadoop/file.txt
```

```
hdfs dfs -tail /user/hadoop/file.txt Last 1KB
```

Count files and directories

```
hdfs dfs -count /user/hadoop
```

3. Permissions and Ownership

Change permissions

```
hdfs dfs -chmod 755 /user/hadoop/file.txt
```

```
hdfs dfs -chmod -R 755 /user/hadoop/directory
```

Change owner

```
hdfs dfs -chown hadoop:hadoop /user/hadoop/file.txt
```

```
hdfs dfs -chown -R hadoop:hadoop /user/hadoop/directory
```

4. Disk Usage

Show disk usage

```
hdfs dfs -du /user/hadoop
```

```
hdfs dfs -du -h /user/hadoop Human-readable
```

```
hdfs dfs -du -s /user/hadoop Summary
```

Show space usage summary

```
hdfs dfs -df -h /
```

HDFS Debug Commands (fsck)

1. File System Check

Check file system health

```
hdfs fsck /
```

Check specific directory

```
hdfs fsck /user/hadoop
```

Detailed block information

```
hdfs fsck /user/hadoop -files -blocks -locations
```

Find corrupt files

```
hdfs fsck / | grep CORRUPT
```

2. Block Operations

List all blocks

```
hdfs fsck /user/hadoop -files -blocks
```

1. Node Management

List all nodes

```
yarn node -list
```

Show node status
yarn node -status <nodeId>

List applications
yarn application -list

Kill application
yarn application -kill <applicationId>

2. Resource Manager Admin

Refresh nodes
yarn rmadmin -refreshNodes

Refresh queues
yarn rmadmin -refreshQueues

Copy between clusters
hadoop distcp hdfs://nn1:8020/source hdfs://nn2:8020/destination

4. Version Information

Check Hadoop version
hadoop version
hdfs version

4. Hadoop vs Linux Commands

File Operations Comparison

Operation	Linux Command	Hadoop Command
List files	ls -l	hdfs dfs -ls
List recursively	ls -R	hdfs dfs -ls -R
Create directory	mkdir dirname	hdfs dfs -mkdir /dirname
Create nested dirs	mkdir -p dir/subdir	hdfs dfs -mkdir -p /dir/subdir
Remove file	rm filename	hdfs dfs -rm /filename
Remove directory	rm -r dirname	hdfs dfs -rm -r /dirname
Copy file	cp source dest	hdfs dfs -cp /source /dest
Move/Rename	mv source dest	hdfs dfs -mv /source /dest

View file content	cat filename	hdfs dfs -cat /filename
View end of file	tail filename	hdfs dfs -tail /filename
View beginning	head filename	hdfs dfs -head /filename

File Information Comparison

Operation	Linux Command	Hadoop Command
File statistics	stat filename	hdfs dfs -stat /filename
Disk usage	du -h	hdfs dfs -du -h
Disk free space	df -h	hdfs dfs -df -h
Word count	wc filename	hdfs dfs -count /filename
File checksum	md5sum filename	hdfs dfs -checksum /filename

Permission Operations Comparison

Operation	Linux Command	Hadoop Command
Change permissions	chmod 755 file	hdfs dfs -chmod 755 /file
Change owner	chown user:group file	hdfs dfs -chown user:group /file
Change group	chgrp group file	hdfs dfs -chgrp group /file
Recursive chmod	chmod -R 755 dir	hdfs dfs -chmod -R 755 /dir

Data Transfer Comparison

Operation	Linux Command	Hadoop Command
Upload file	cp /local/file /remote/	hdfs dfs -put /local/file /hdfs/
Download file	cp /remote/file /local/	hdfs dfs -get /hdfs/file /local/
Append to file	cat file >> existing	hdfs dfs -appendToFile local /hdfs/file

Key Differences

1. Storage Architecture

Linux:

- Single machine file system
- Direct disk access
- Immediate read/write operations

Hadoop:

- Distributed file system across multiple nodes
- Data split into blocks (default 128MB)
- Replicated across multiple DataNodes (default 3 replicas)

2. Command Prefix

Linux:

- Direct commands: ls, cat, cp

Hadoop:

- Requires prefix: hdfs dfs -ls, hdfs dfs -cat, hdfs dfs -cp
- Alternative: hadoop fs -ls (older syntax, still works)

3. File Paths

Linux:

- Absolute: /home/user/file.txt
- Relative: ./file.txt or file.txt

Hadoop:

- Always uses / as root
- Full URI: hdfs://namenode:port/path
- Shorthand: /user/hadoop/file.txt

4. File Access Speed

Linux:

- Millisecond access times
- Suitable for small files

Hadoop:

- Higher latency (seconds)
- Optimized for large files
- Not suitable for small files (small files problem)

5. Data Locality

Linux:

- Data on single machine

Hadoop:

- Data distributed across cluster
- Computation moves to data (MapReduce principle)

6. Permissions Model

Linux:

- Standard POSIX permissions (user, group, others)
- Additional ACLs available

Hadoop:

- Similar to Linux permissions
- Simplified ACL support
- Additional HDFS-specific permissions

7. File Editing

Linux:

- Files can be edited in place
- Random read/write access

Hadoop:

- Write-once-read-many (WORM) model
- Cannot modify existing files (can append in some cases)
- Must delete and recreate to modify

Commands Not Available in Hadoop:

Some Linux commands don't have direct equivalents in Hadoop:

Not Available:

- grep (use MapReduce or Spark instead)
- sed, awk (use data processing frameworks)
- find with complex predicates (limited functionality)
- ln (symbolic links)

Hadoop-Specific Commands (No Linux Equivalent)

NameNode operations

`hdfs namenode -format`

`hdfs dfsadmin -safemode enter`

HDFS health check

`hdfs fsck /`

Cluster balancing

`hdfs balancer`

Replication management

`hdfs dfs -setrep 3 /file.txt`

Practice Exercises

Exercise 1: Namespace Operations

1. Create a directory structure in HDFS
2. Set name quota of 100 files
3. Set space quota of 1GB
4. Try exceeding quotas and observe behavior

Exercise 2: Cluster ID Investigation

1. Find VERSION file on your cluster
2. Record the Cluster ID
3. Explain each field in the VERSION file
4. Compare VERSION files across nodes

Exercise 3: Command Practice

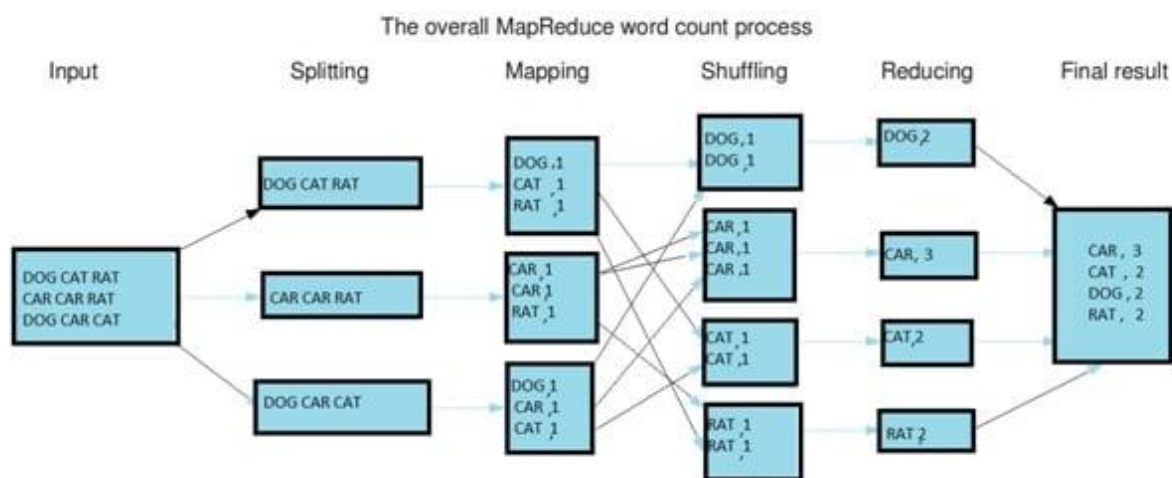
1. Upload 10 files to HDFS
2. Change permissions to 755
3. Create a backup copy

4. Check file system health with fsck
5. Generate cluster report

Exercise 4: Linux to Hadoop Migration

1. Write a script using Linux commands
2. Convert it to use Hadoop commands
3. Compare execution time and behavior
4. Identify limitations of each approach

Map Reduce:



MapReduce Tutorial - Complete Notes

What is MapReduce?

Definition:

- Processing engine of Apache Hadoop derived from Google MapReduce
- Written primarily in Java
- Processes huge amounts of data through mapping and reducing steps
- Converts data into key/value pairs (tuples) for processing

Core Process:

1. **Mapping Step:** Converts input data into another set of data by breaking elements into key/value pairs
 2. **Reducing Step:** Combines mapped data tuples into a smaller set of tuples
-

Key Characteristics

Feature	Details
Chief Phases	Mapping & Reducing
Data Caching	Hard disk
Hadoop Dependency	Yes (requires Hadoop)
Processing Speed	Average
Scalability	Single node to thousands of nodes

How MapReduce Works

Core Principles:

- **Massively parallel processing** framework
- Scales over commodity hardware
- **Computation moves to data** (not data to computation)
- Reduces network congestion and improves efficiency
- Configuration changes allow scaling to larger datasets

Key Benefits:

Enables parallel processing for Big Data jobs

Applicable to wide variety of business applications

Cost-effective compared to centralized processing

Can integrate with SQL for parallel processing

MapReduce Architecture

Processing Stages:

INPUT (HDFS) → MAPPING → SHUFFLING → SORTING → REDUCING → OUTPUT (HDFS)

1. Mapping Stage

- First step in MapReduce process
- Reads data from HDFS (Hadoop Distributed File System)
- Input: Directory or file
- Process:
 - Data file fed into mapper function line by line
 - Mapper processes and reduces into smaller data blocks

2. Shuffling Process

- Transfers data from mapper to reducer
- Can start before mapping completes
- Without shuffling, no input to reducer phase

3. Sorting Process

- Data sorted to reduce processing time
- Provides cue when next key differs from previous
- Helps reducer identify distinct keys

4. Reducing Stage

- Multiple processes possible
- Takes specific key-value pairs as input
- Calls reduce function on key-value pairs
- Output stored directly in HDFS

MapReduce Terminologies version 1

Term	Definition
------	------------

-----	-----
-------	-------

MasterNode	Where JobTracker runs; accepts job requests from clients
------------	--

SlaveNode	Where mapping and reducing programs execute
-----------	---

JobTracker	Schedules jobs and tracks assigned jobs using TaskTracker
------------	---

TaskTracker	Tracks tasks and reports status to JobTracker
-------------	---

Job	Execution of Mapper & Reducer across entire dataset
-----	---

Task	Execution of Mapper & Reducer on specific data section
------	--

TaskAttempt	Specific task execution attempt on a SlaveNode
-------------	--

Problems Solved by MapReduce

Origin:

- Derived from Google MapReduce
- Created for parsing large web pages
- Delivers search results based on keywords

Solutions Provided:

1. **Simplified Big Data Processing**
 - Previously herculean task to parse huge data
 - Breaks data into manageable chunks
 2. **Easy Programming**
 - Write in Java, Python, or C++
 - Widespread adoption possible
 3. **High Scalability**
 - Works on entire Hadoop clusters
 - Runs on commodity hardware
 4. **Fault-Tolerant**
 - Continues working even if nodes fail
 - Data stored in multiple locations
 5. **Efficient Processing**
 - Computation moves to data location
 - Reduces I/O time
 - Increases processing speeds
-

Major Companies Using MapReduce

- Amazon
- Facebook

- Google
 - Microsoft
 - Yahoo
 - General Electric
 - IBM
-

YARN Architecture -

What is YARN?

YARN = Yet Another Resource Negotiator

Introduction:

- Introduced in **Hadoop 2.0** (2013)
- Separates resource management from processing logic
- Enables Hadoop to support multiple processing frameworks
- Overcomes limitations of Hadoop 1.x MapReduce

Key Innovation:

Hadoop 1.x: JobTracker = Resource Management + Job Scheduling

Hadoop 2.x: YARN = Resource Management (separate) + Application Management

...

Why YARN Was Created

Problems with Hadoop 1.x:

Scalability Bottleneck

- Limited to ~4,000 nodes
- JobTracker became overwhelmed

Single Point of Failure

- If JobTracker fails, entire cluster stops

Resource Utilization Issues

- Fixed map and reduce slots

- Slots couldn't be shared between map/reduce
- Led to resource wastage

Only MapReduce Supported

- No other processing frameworks possible
- Couldn't run Spark, Flink, etc.

Tight Coupling

- Resource management and job scheduling tightly coupled
- Hard to maintain and scale

YARN Solutions:

Scalability to 10,000+ nodes

High Availability (no single point of failure)

Better resource utilization

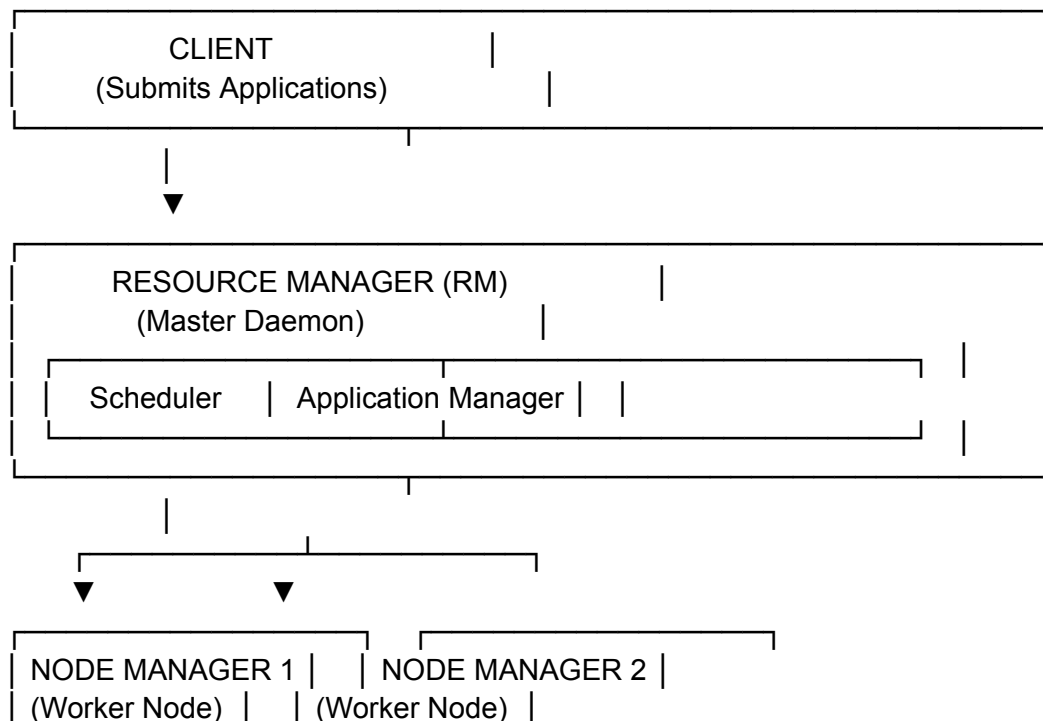
Support for multiple frameworks

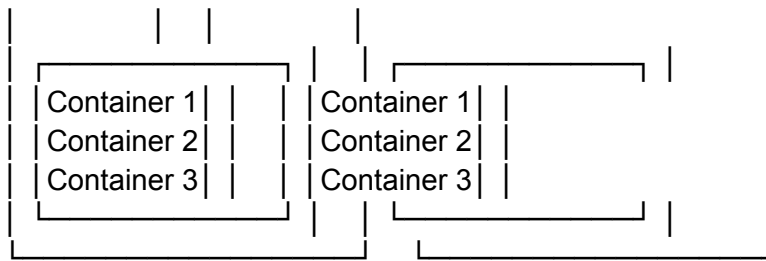
Separation of concerns

YARN Architecture Overview

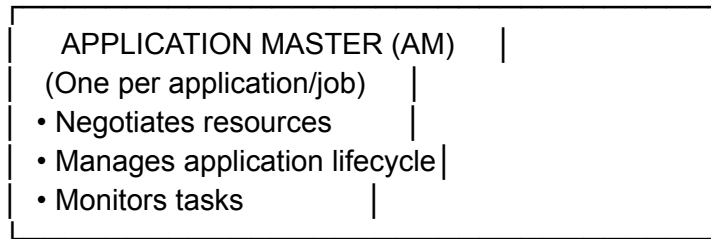
High-Level Architecture:

...





Per-Application Level:



...

YARN Components

1. Resource Manager (RM)

Location: Master node (one per cluster)

Role: Global resource manager and scheduler

Two Main Components:

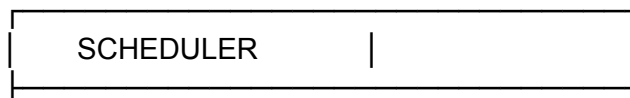
a) Scheduler

- Allocates resources to applications
- Pure scheduler (doesn't monitor or track status)
- Pluggable scheduling policies
- Doesn't restart failed tasks

Scheduling Policies:

- FIFO Scheduler: First In First Out (simple)
- Capacity Scheduler: Multi-tenant, capacity guarantees
- Fair Scheduler: Fair share of resources

...



• Resource Allocation	
• Queue Management	
• Capacity Management	
• Priority Handling	

...

b) Application Manager (ASM)

- Accepts job submissions
- Negotiates first container for Application Master
- Monitors Application Masters
- Restarts Application Master on failure

...

APPLICATION MANAGER	
• Accept Applications	
• Launch App Master	
• Monitor App Master	
• Restart on Failure	

...

Key Responsibilities:

- Manage cluster resources
- Track node health
- Maintain resource availability
- Handle client requests
- Security and authentication

2. Node Manager (NM)

Location: Worker nodes (one per node)

Role: Per-node agent managing resources and containers

Key Responsibilities:

...

NODE MANAGER	
• Launch Containers	

• Monitor Containers	
• Track Resources	
• Report to RM	
• Kill Containers	
• Log Management	

...

Functions:

- Launches and monitors containers
- Manages local resources (CPU, memory, disk, network)
- Sends heartbeats to Resource Manager
- Reports resource availability
- Enforces resource limits
- Handles container lifecycle

Container Management:

- CPU cores allocation
- Memory allocation
- Disk space management
- Network bandwidth (in some versions)

Heartbeat Information:

- Node health status
- Container status
- Available resources
- Running applications

3. Application Master (AM)

Location: Runs in a container on a worker node

Role: Per-application/job coordinator

Key Characteristics:

- One per application/job
- Application-specific
- Framework-specific (MapReduce AM, Spark AM, etc.)
- Runs as a container

Responsibilities:

...

APPLICATION MASTER
• Request Resources • Negotiate Containers • Monitor Tasks • Handle Failures • Report Progress • Application Lifecycle

...

Detailed Functions:

1. Resource Negotiation:
 - Requests containers from RM
 - Specifies resource requirements (CPU, memory)
 - Gets container allocation from RM
2. Task Management:
 - Launches tasks in containers
 - Monitors task progress
 - Handles task failures
 - Implements task retry logic
3. Progress Tracking:
 - Tracks application progress
 - Reports status to RM
 - Provides information to clients
4. Fault Tolerance:
 - Detects task failures
 - Requests replacement containers
 - Re-executes failed tasks

Example Application Masters:

- MRAppMaster (for MapReduce)
- Spark Application Master
- Flink Application Master
- Tez Application Master

4. Container

Definition: Abstract representation of resources

What is a Container?

- Basic unit of resource allocation
- Encapsulates resources on a single node
- Granted by Resource Manager
- Managed by Node Manager

Container Contents:

...

CONTAINER	
• CPU Cores (vCores)	
• Memory (RAM in MB/GB)	
• Disk Space	
• Network Bandwidth	
• Process/Task Code	

...

Container Characteristics:

- Resource allocation unit
- Runs single task/process
- Has resource limits enforced by NM
- Short-lived (task duration)
- Location-aware

Container Lifecycle:

...

Request → Allocate → Launch → Execute → Complete → Release

...

Example:

...

Container_1: 2 vCores, 4GB RAM → Runs Map Task

Container_2: 1 vCore, 2GB RAM → Runs Reduce Task

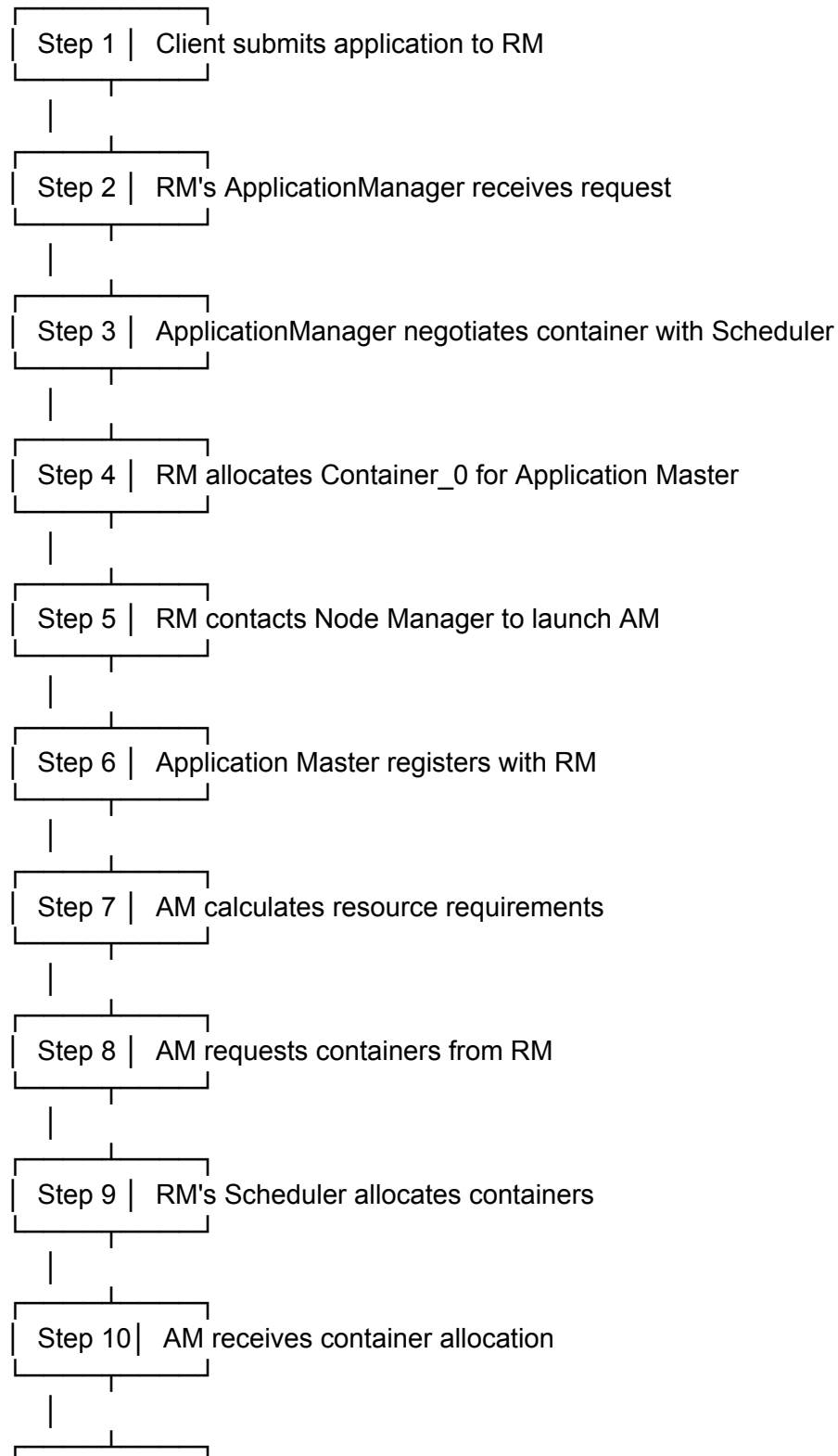
Container_3: 4 vCores, 8GB RAM → Runs Application Master

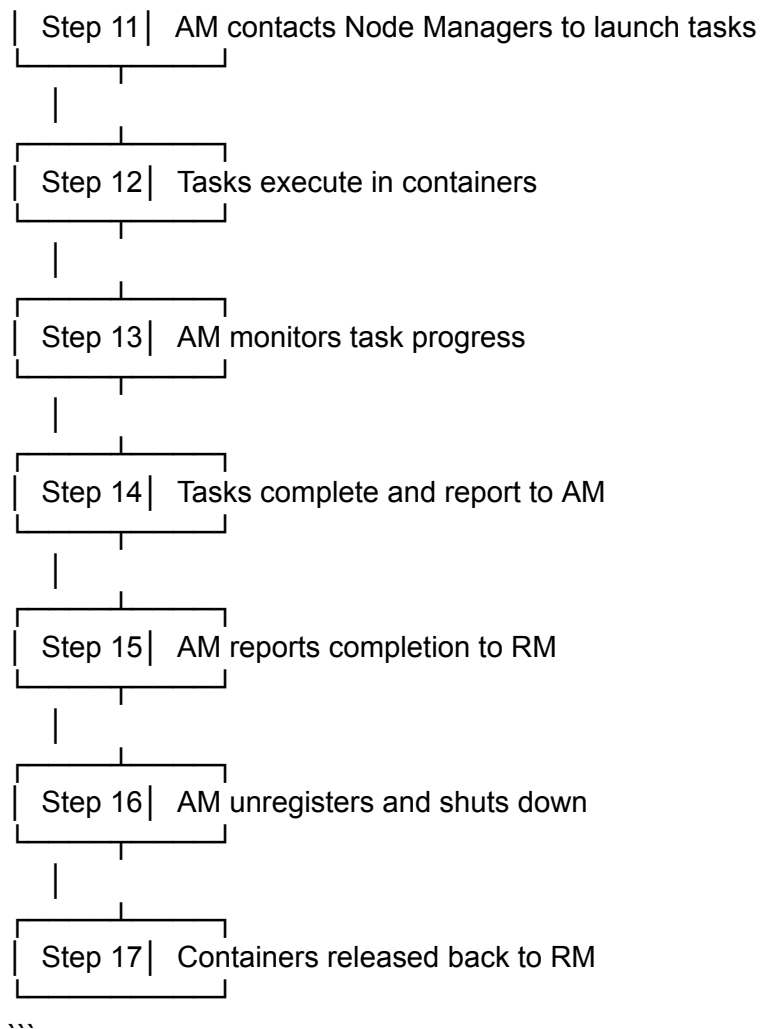
...

YARN Workflow

Complete Application Execution Flow:

...





Detailed Step-by-Step Execution:

Phase 1: Application Submission

Step 1-2: Client Submission

...

Client → Resource Manager

"I want to run a MapReduce job"

...

- Client submits application to RM
- Includes: job configuration, input/output paths, JAR files
- ApplicationManager receives submission

Step 3-5: Launch Application Master

...

Resource Manager → Node Manager

"Launch Application Master in Container_0"

...

- RM allocates first container for AM
- Chooses suitable Node Manager
- NM launches AM container
- AM process starts running

Phase 2: Resource Negotiation

Step 6: Registration

...

Application Master → Resource Manager

"I'm alive and ready, here's my info"

...

- AM registers with RM
- Provides tracking URL
- RM acknowledges registration

Step 7-8: Resource Request

...

Application Master → Resource Manager

"I need 10 containers: 5 with 2GB RAM, 5 with 4GB RAM"

...

- AM calculates requirements based on job
- Requests specific container resources
- Can specify locality preferences

Step 9-10: Container Allocation

...

Resource Manager → Application Master

"Here are 10 containers on these nodes"

...

- Scheduler finds available resources
- Allocates containers matching requirements
- Considers data locality
- Sends container tokens to AM

Phase 3: Task Execution

Step 11: Task Launch

...

Application Master → Node Managers

"Launch these tasks in allocated containers"

...

- AM contacts NMs with container tokens
- Provides task code and dependencies
- NMs start containers and launch tasks

Step 12-13: Execution and Monitoring

...

Tasks → Application Master

"Progress: 25% complete..."

...

- Tasks execute in containers
- Send progress updates to AM
- AM aggregates progress information
- AM reports to RM periodically

Phase 4: Completion

Step 14-17: Cleanup

...

Tasks → Application Master → Resource Manager

"All tasks completed successfully"

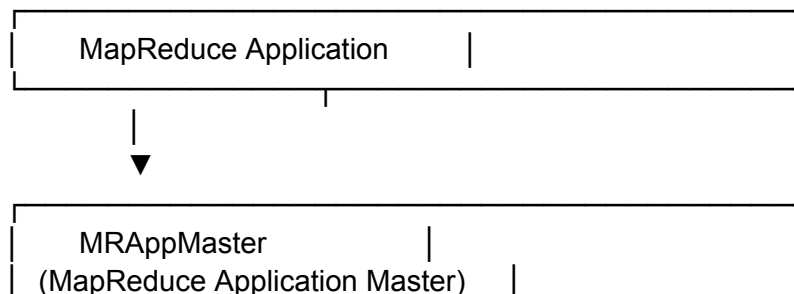
...

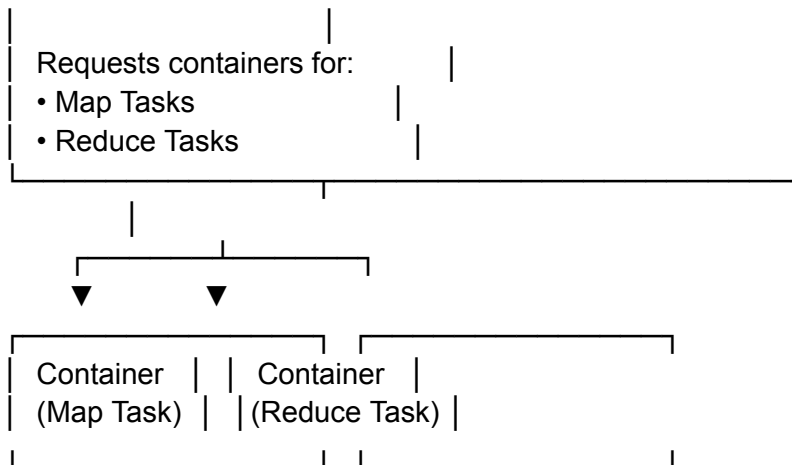
- Tasks finish and report to AM
- AM verifies all tasks completed
- AM reports final status to RM
- AM unregisters from RM
- Containers released
- Resources returned to cluster pool

MapReduce on YARN

How MapReduce Runs on YARN:

...





MRAppMaster Responsibilities:

- Requests containers for map/reduce tasks
- Manages MapReduce job lifecycle
- Handles data shuffling coordination
- Implements speculative execution
- Manages task failures and retries

Resource Allocation in YARN

Container Resource Specification:

yaml

Container Request:

- vCores: 2 *CPU cores*
- Memory: 4096 MB *RAM*
- Priority: 1 *Request priority*
- Locality: Node_5 *Preferred node (optional)*
- Relax Locality: true *Allow other nodes if preferred unavailable*

Resource Configuration:

yarn-site.xml settings:

xml

<!-- Maximum memory per container -->

<property>

<name>yarn.scheduler.maximum-allocation-mb</name>

```

    <value>8192</value>
  </property>

  <!-- Minimum memory per container -->
  <property>
    <name>yarn.scheduler.minimum-allocation-mb</name>
    <value>1024</value>
  </property>

  <!-- Maximum CPU cores per container -->
  <property>
    <name>yarn.scheduler.maximum-allocation-vcores</name>
    <value>4</value>
  </property>

  <!-- Node Manager resources -->
  <property>
    <name>yarn.nodemanager.resource.memory-mb</name>
    <value>16384</value>
  </property>

  <property>
    <name>yarn.nodemanager.resource.cpu-vcores</name>
    <value>8</value>
  </property>
  ...

```

YARN Scheduling Policies

1. FIFO Scheduler

```

...
Queue: [Job1] → [Job2] → [Job3] → [Job4]
      (runs first, gets all resources)
...

```

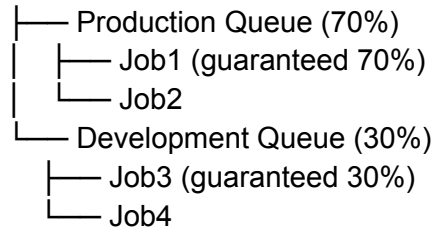
Characteristics:

- Simple First-In-First-Out
- Single queue
- No preemption
- Not suitable for multi-tenant clusters

2. Capacity Scheduler

...

Root Queue (100%)



...

Characteristics:

- Multiple queues with capacity guarantees
- Hierarchical queue structure
- Each queue gets minimum capacity
- Can use unused capacity from other queues
- Elasticity and sharing

Use Case: Multi-tenant environments with SLAs

3. Fair Scheduler

...

Time 0-5s: Job1 gets 100% resources

Time 5-10s: Job1 gets 50%, Job2 gets 50%

Time 10-15s: Job1 gets 33%, Job2 gets 33%, Job3 gets 33%

...

Characteristics:

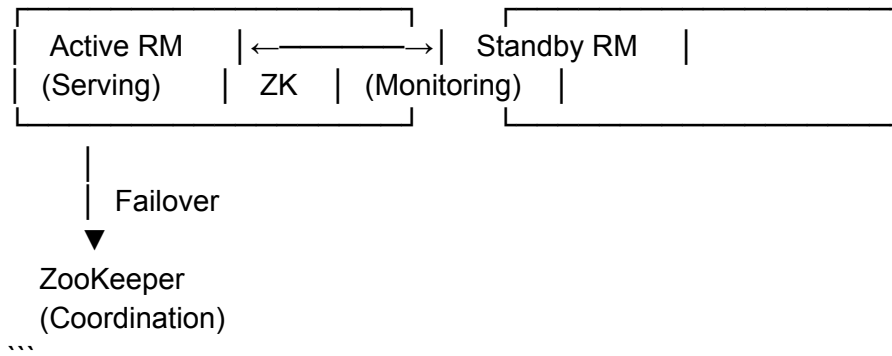
- Equal share of resources over time
- Preemption support
- Multiple pools/queues
- Dynamic fair sharing

Use Case: Shared clusters where fairness is priority

YARN High Availability (HA)

Resource Manager HA:

...



Features:

- Active/Standby RM configuration
- Automatic failover using ZooKeeper
- No single point of failure
- State store for recovery

State Storage:

- HDFS or ZooKeeper
- Stores application state
- Enables recovery after failover

Advantages of YARN

✓ Scalability

- Supports 10,000+ nodes
- Linear scaling capability

✓ Multi-Tenancy

- Multiple users and applications
- Resource isolation and sharing

✓ Flexibility

- Supports various processing models
- Not limited to MapReduce

✓ Resource Utilization

- Dynamic resource allocation
- No fixed slots
- Better utilization than Hadoop 1.x

✓ Backward Compatibility

- MapReduce jobs run without modification
- Gradual migration path

✓ High Availability

- No single point of failure
- Automatic failover

✓ Better Management

- Separation of concerns
- Easier to maintain and debug

YARN vs Hadoop 1.x Comparison

Feature	Hadoop 1.x	YARN (Hadoop 2.x/3.x)
Master Daemon	JobTracker	Resource Manager
Worker Daemon	TaskTracker	Node Manager
Resource Unit	Fixed slots	Flexible containers
Scalability	~4,000 nodes	10,000+ nodes
Frameworks	MapReduce only	Multiple (MR, Spark, Flink, etc.)
HA Support	No	Yes
Resource Management	Slot-based	Container-based
Single Point of Failure	Yes (JobTracker)	No (RM HA)
Job Monitoring	JobTracker	Application Master
Resource Utilization	Poor (fixed slots)	Excellent (dynamic)

Supported Frameworks on YARN

YARN can run multiple processing frameworks:

...

YARN			
MapReduce	Spark	Flink	
Tez	Storm	Giraph	
Hive	Pig	HBase	

...

Popular Frameworks:

- MapReduce - Batch processing
- Apache Spark - In-memory processing
- Apache Flink - Stream processing
- Apache Tez - DAG-based execution
- Apache Storm - Real-time streaming
- Apache Giraph - Graph processing

YARN Configuration Files

Key Configuration Files:

1. yarn-site.xml

- YARN-specific configurations
- Resource Manager settings
- Node Manager settings
- Container configurations

2. core-site.xml

- Hadoop core settings
- Filesystem configurations

3. mapred-site.xml

- MapReduce on YARN settings
- MRAppMaster configurations

4. capacity-scheduler.xml

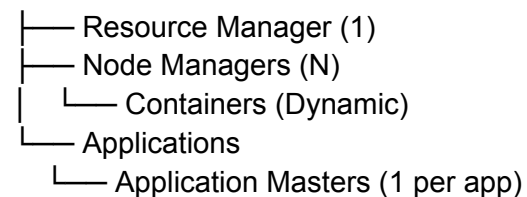
- Capacity scheduler configuration
- Queue definitions

Key YARN Concepts Summary

Resource Model:

...

Cluster



...

Resource Hierarchy:

...

vCores + Memory = Container

Containers = Node Manager Resources

Node Managers = Cluster Resources

...

Communication Pattern:

...

Client $\longleftrightarrow$ Resource Manager

Resource Manager $\longleftrightarrow$ Node Managers

Application Master $\longleftrightarrow$ Resource Manager

Application Master $\longleftrightarrow$ Node Managers

Important YARN Ports

Service	Default Port	Purpose
Resource Manager Web UI	8088	RM web interface
Resource Manager	8032	Client to RM
Resource Manager Scheduler	8030	AM to RM
Node Manager Web UI	8042	NM web interface
Node Manager	8040	Container communication
MapReduce JobHistory	19888	Job history server

YARN Monitoring and Management

Web UIs:

Resource Manager UI (port 8088):

- Cluster metrics
- Application status
- Queue information
- Node health

Node Manager UI (port 8042):

- Container logs
- Node resources
- Local applications

Application Master UI:

- Application-specific monitoring
- Task progress
- Job details

CLI Commands:

List applications

yarn application -list

Kill application

yarn application -kill <app_id>

View application logs

yarn logs -applicationId <app_id>

Node status

yarn node -list

Queue status

yarn queue -status <queue_name>

Best Practices for YARN

1. Resource Configuration:

- Calculate container sizes based on workload
- Leave headroom for OS and other processes
- Don't over-allocate memory

2. Queue Management:

- Design queue hierarchy carefully
- Set appropriate capacity guarantees

- Configure max capacity limits

3. Monitoring:

- Monitor Resource Manager regularly
- Track container allocation
- Watch for resource bottlenecks

4. High Availability:

- Always enable RM HA in production
- Configure proper ZooKeeper ensemble
- Test failover scenarios

5. Security:

- Enable Kerberos authentication
 - Configure ACLs for queues
 - Use encryption for data transfer
-

Summary

YARN Core Principles:

1. **Separation of Concerns:** Resource management separated from application logic
2. **Scalability:** Horizontal scaling to thousands of nodes
3. **Multi-Framework:** Support for diverse processing models
4. **Resource Efficiency:** Dynamic container-based allocation
5. **Fault Tolerance:** HA and automatic recovery

Key Takeaway: YARN transformed Hadoop from a **MapReduce-only platform** into a **general-purpose distributed computing framework**, enabling the modern Big Data ecosystem.

PART 1: What is Apache Hive?

Introduction

Apache Hive is a data warehouse software built on top of Hadoop that allows you to read, write, and manage large datasets using SQL-like queries.

Key Points to Cover:

1. The Problem Hive Solves

- Hadoop uses MapReduce (Java code) - complex for analysts who know SQL
- Writing MapReduce programs is time-consuming and requires programming skills
- Need: A way to query big data using familiar SQL syntax

2. What Hive Does

- Provides SQL-like interface called **HiveQL** (Hive Query Language)
- Translates SQL queries into MapReduce jobs automatically
- Manages large datasets stored in Hadoop Distributed File System (HDFS)

3. Key Characteristics

- **Not a database:** It's a data warehouse solution
- **Best for:** Batch processing, analytics, and large-scale data analysis
- **Not suitable for:** Real-time queries or frequent updates (OLTP)
- **Schema on Read:** Unlike traditional databases with schema on write

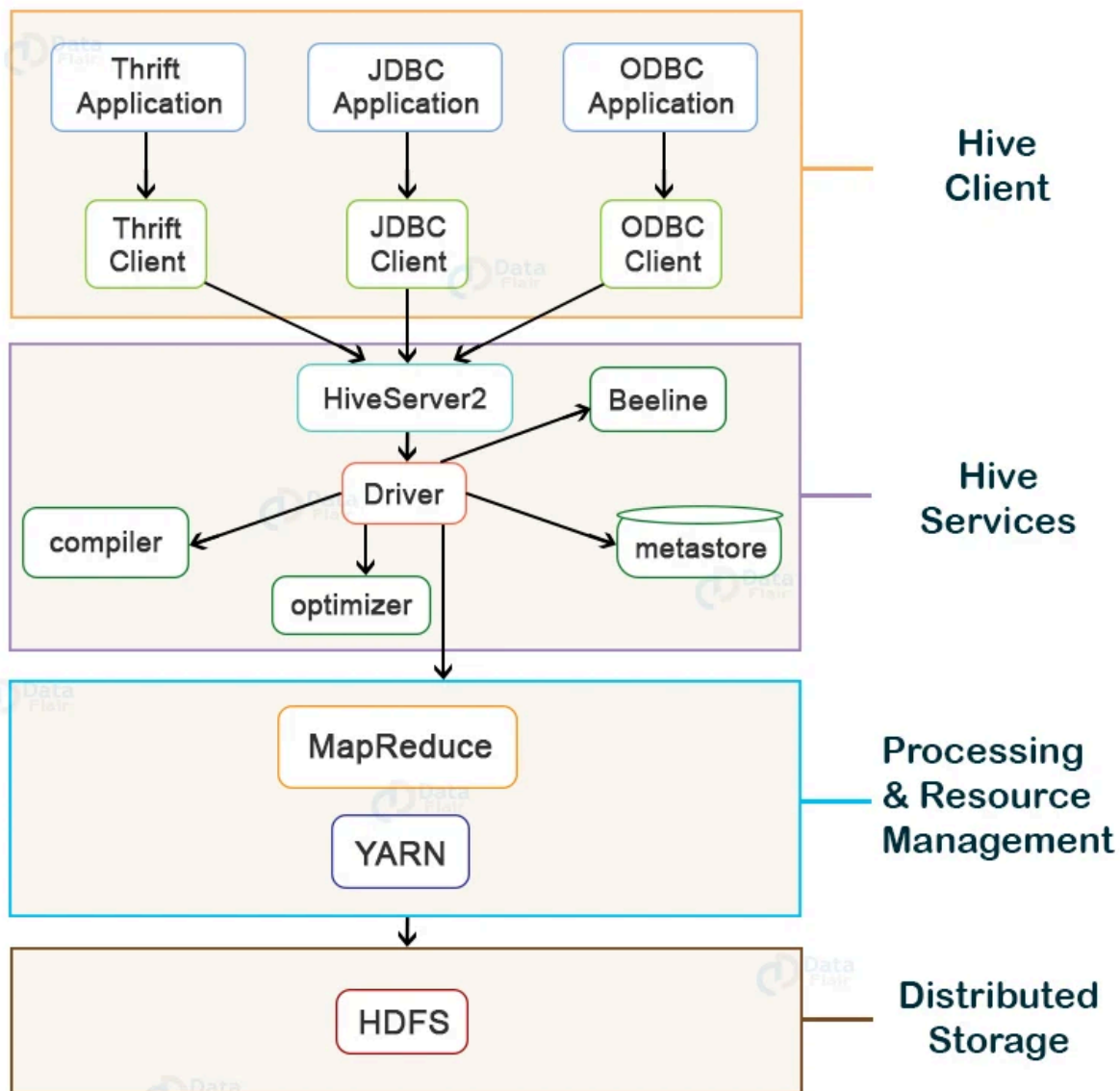
4. Who Uses Hive?

- Data analysts who know SQL but not MapReduce
- Data scientists analyzing large datasets
- Business intelligence professionals

5. Language Support

- Supports multiple programming languages: Python, Java, C++, Ruby
- Uses standard protocols: JDBC, ODBC, Thrift drivers

PART 2: Hive Architecture



Hive Architecture & Its Components

Component 1: Hive Clients (How users connect)

Users can connect to Hive in three ways:

a) Thrift Clients

- Based on Apache Thrift protocol
- Allows cross-platform communication
- Supports multiple programming languages

b) JDBC Client

- For Java applications
- Uses Thrift to communicate with Hive Server
- Familiar interface for Java developers

c) ODBC Client

- For applications using ODBC protocol
 - Also uses Thrift underneath
 - Common for BI tools
-

Component 2: Hive Services (The Engine)

These are the core services that process your queries:

1. Beeline

- Command-line interface (like MySQL command line)
- Where users type and submit queries
- Works with HiveServer2

2. HiveServer2

- Handles client requests
- **Key improvement:** Supports multiple concurrent clients (unlike HiveServer1)
- Provides JDBC/ODBC support

3. Hive Driver

- **First stop** for every query
- Creates session handles
- Sends queries to the compiler

4. Hive Compiler

- **Parses** the HiveQL query
- Performs **semantic analysis** (checks if query makes sense)
- **Type-checking** using metadata from Metastore
- Generates an **execution plan** (DAG - Directed Acyclic Graph)

5. Optimizer

- Takes the execution plan and **improves it**
- Splits tasks for better efficiency
- Makes queries run faster

6. Execution Engine

- **Executes** the optimized plan
- Submits jobs to Hadoop/MapReduce
- Manages dependencies between stages

7. Metastore (Critical Component)

- **Central repository** for all metadata
- Stores information about:
 - Table structures
 - Column names and types
 - Partitions
 - File locations in HDFS
 - Serializer/Deserializer info

8. HCatalog

- Table and storage management layer
- Allows other tools (Pig, MapReduce) to access Hive tables
- Built on top of Metastore

9. WebHCat

- REST API for HCatalog
- HTTP interface for metadata operations
- Run jobs via web requests

Component 3: Processing Framework

- Uses **MapReduce** as the execution engine

- Splits data into chunks
 - Processes in parallel across cluster
-

Component 4: Distributed Storage

- Built on **Hadoop HDFS**
 - Data stored across distributed nodes
 - Fault-tolerant and scalable
-

PART 3: Hive Workflow

SELECT name, age FROM employees WHERE age > 30;

STEP 1: Execute Query

- Who: User Interface (Beeline/JDBC client)
- What: User submits the query
- Action: Calls execute interface to the Driver

STEP 2: Get Plan

- Who: Driver
- What: Receives the query
- Actions
 - Creates a session handle
 - Sends query to Compiler
 - Requests execution plan

STEP 3: Get Metadata

- Who: Compiler
- What: Needs information about the "employees" table
- Action: Sends metadata request to Metastore
- Why: To understand table structure, column types, data location

STEP 4: Send Metadata

- Who: Metastore

- What: Retrieves stored metadata
- Action: Sends metadata back to Compiler
- Info Sent: Table schema, column types, partition info, file locations

STEP 5: Compilation & Planning

- Who: Compiler
- What: Creates execution plan
- Process:
 1. Type-checking: Ensures "age > 30" is valid (age is numeric)
 2. Semantic analysis: Validates query structure
 3. Generates DAG: Creates execution plan with:
 - Map operator trees (for mapper tasks)
 - Reduce operator trees (for reducer tasks)

STEP 6: Send Plan

- Who: Compiler
- What: Completed execution plan (DAG)
- Action: Sends optimized plan to Driver

STEP 7: Execute Plan

- Who: Driver
- What: Receives execution plan
- Action: Forwards plan to Execution Engine

STEP 8: Submit Job to MapReduce

- Who: Execution Engine
- What: Breaks down DAG into stages
- Actions:
 1. Submits stages to MapReduce/Hadoop
 2. Each stage runs as a separate MapReduce job
 3. Deserializer reads data from HDFS files
 4. Data passes through operator trees (filters, transformations)
 5. Serializer writes output to temporary HDFS files
 6. Temporary files feed into next stages

STEP 9: Process Data

- MapReduce Jobs Execute
 - Mappers: Read and filter data (WHERE age > 30)
 - Reducers: Aggregate if needed
 - Output stored in temporary HDFS files

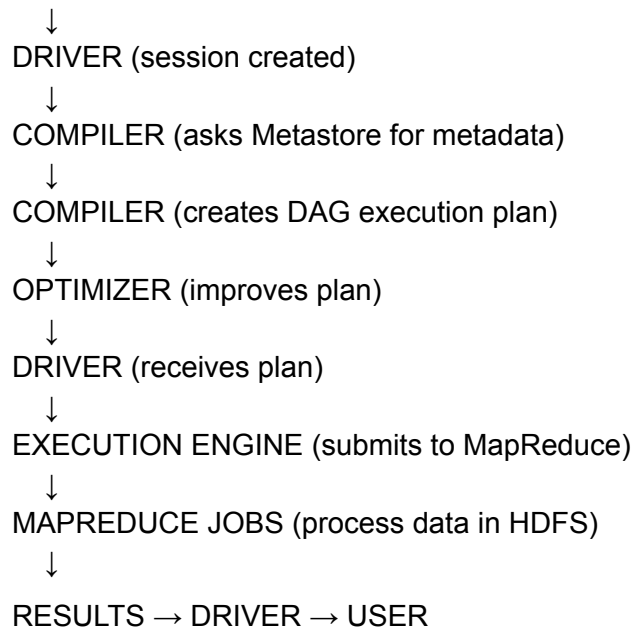
STEP 10: Return Results

- Who: Execution Engine → Driver → User Interface
- What: Query results
- Actions:
 - For queries (SELECT): Engine reads temporary files from HDFS
 - Driver fetches results
 - Results sent back to Hive interface (Beeline/client)
 - User sees output
- For DML operations (INSERT/UPDATE): Temporary files moved to table's permanent location

Visual Summary for Students

...

USER QUERY



What is DAG (Directed Acyclic Graph)?

Great question! Let me explain DAG in the context of Hive.

Simple Definition

DAG = Directed Acyclic Graph

It's a visual representation of tasks/operations where:

- **Directed:** Arrows show the flow/direction (one task leads to another)
 - **Acyclic:** No cycles/loops (you can't go back to a previous step)
 - **Graph:** A diagram showing nodes (tasks) connected by edges (dependencies)
-

Breaking Down the Terms

1. Graph

- A structure with **nodes** (points) and **edges** (lines connecting them)
- Like a flowchart or map

2. Directed

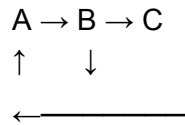
- Arrows show direction: Task A → Task B
- You can only move in one direction
- Task B depends on Task A completing first

3. Acyclic

- **No cycles** = No circular paths
 - You can't return to a task you already completed
 - No infinite loops
-

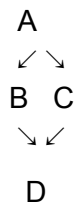
Visual Example

NOT a DAG (has a cycle):



This creates a loop: $A \rightarrow B \rightarrow C \rightarrow A$ (circular dependency)

IS a DAG:



Tasks flow in one direction, no going back

DAG in Hive Context

When you submit a HiveQL query, the **Compiler creates a DAG as the execution plan.**

Example Query:

```
SELECT department, AVG(salary)
FROM employees
WHERE age > 30
GROUP BY department;
```

The DAG Execution Plan:

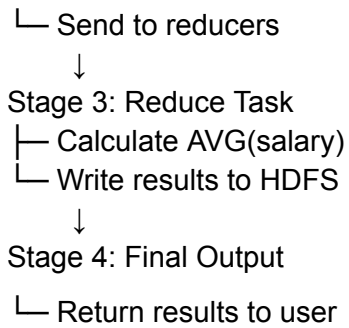
Stage 1: Map Task

- |— Read data from HDFS
- |— Filter: WHERE age > 30
- |— Output to temp file

↓

Stage 2: Shuffle/Sort

- |— Group by department



Each stage is a node, and arrows show dependencies.

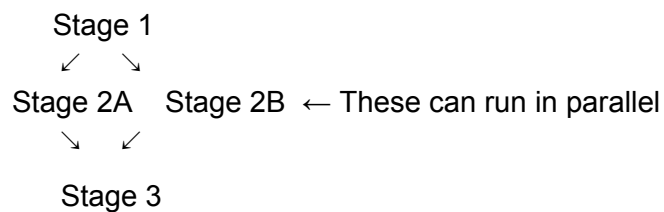
Why DAG in Hive?

1. Shows Dependencies

- Stage 2 cannot start until Stage 1 completes
- Stage 3 needs output from Stage 2
- Clear execution order

2. Parallel Execution

If there are independent stages:



3. Optimization

- Optimizer can rearrange independent tasks
- Can identify which stages can run simultaneously
- Improves efficiency
-